

NORTHERN SHIFT GUARD

Project Report — Cursor Hackathon Sudbury 2026

Team: NorthMind | Track: Mining & Industrial Innovation

Report generated: June 27, 2026 19:56 UTC

Category	Details
Project	Northern Shift Guard — AI shift-start PPE & fatigue screening
Team	NorthMind
Event	Cursor Hackathon Sudbury 2026 — Build the North
Track	Mining & Industrial Innovation
Languages	Python (backend) · TypeScript (frontend)
Core AI	GPT-4o Vision + NVIDIA Nemotron (GPT-4o-mini fallback)
Database	TiDB Cloud (MySQL) · SQLite local fallback
Frontend	React + Vite + Tailwind CSS

NOTE ON FILE SUBMISSION: The complete project archive could not be attached to Devpost due to file size limits (venv, node_modules, and sample media exceed 500MB). The full source code and all assets are available on GitHub — see the repository link on the Devpost project page.

TABLE OF CONTENTS

1. Problem Statement
2. Solution Overview
3. System Architecture
4. AI Pipeline Detail
5. Zone-Aware Compliance Engine
6. Tech Stack
7. Project File Structure
8. API Reference
9. Setup & Running
10. Demo Walkthrough
11. Judging Criteria Mapping
12. What We Learned
13. Next Steps
14. Submission Note & GitHub

1. PROBLEM STATEMENT

Northern Ontario mines are losing experienced tradespeople to retirement faster than they can be replaced. New workers bring elevated risk — especially at shift start, when fatigue from overnight travel and unfamiliarity with PPE protocols combine. Shift-start safety checks are still done manually: a supervisor glances at a worker, makes a judgment call, and nothing is recorded. If something goes wrong — an injury, a liability dispute, an MOL inspection — there is no record of what was checked or what action was recommended. The decision is a black box.

Key Pain Points

- No structured record of shift-start PPE checks (MOL audit risk)
- Generic PPE detectors apply the same ruleset to every zone — surface workers flagged for no hard hat when their zone doesn't require one
- Supervisors get a pass/fail label with no rationale — not actionable
- Ontario Regulation 854 specifies zone-level requirements that existing tools ignore

2. SOLUTION OVERVIEW

Northern Shift Guard is an explainable AI shift-start screening tool for Northern Ontario mining operations. It replaces the manual judgment call with a four-layer pipeline that detects, reasons, and records — producing an auditable, zone-specific decision every time.

Feature	Description	Technology
GPT-4o Vision	Detects hard hat, hi-vis vest, visible fatigue cues from worker photo	OpenAI GPT-4o
Zone Compliance	PPE checked against per-zone requirements under Ontario Reg 854	zone_service.py
Nemotron Reasoning	Plain-language supervisor action with priority, rationale, steps	NVIDIA Nemotron
TiDB Audit Trail	Every scan persisted: vision JSON, zone result, Nemotron action, TiDB Scan	TiDB Cloud
Explainable Evidence	Per-region evidence cards show what the model saw and why	Structured JSON

Key differentiator: the same worker photo submitted for **Surface / Yard** produces a different compliance verdict than **Active Stope** — because hard hat is not required at surface but mandatory underground. This is grounded in actual Ontario Reg 854 section references.

3. SYSTEM ARCHITECTURE

Layer	Components	Note
Frontend	Worker Photo Upload · Zone Selector · React Dashboard (compliance panel, evidence cards, audit trail)	React 18
Backend	FastAPI · POST /api/analyze · GET /api/scans · GET /api/zones · Pydantic schema	Python 3.12
AI Layer	GPT-4o Vision · Zone Compliance Engine · NVIDIA Nemotron / GPT-4o-mini fallback	OpenAI · NVIDIA Context
Storage	TiDB Cloud — vision_json, zone_compliance, nemotron_action, provider_info, MySQL for local fallback	

Data Flow

- Supervisor selects mine zone + uploads worker photo via React UI
- React POSTs multipart form (file + zone) to FastAPI /api/analyze
- FastAPI base64-encodes image and calls GPT-4o with structured JSON prompt
- Vision result feeds Zone Compliance Engine (detected PPE vs zone requirements)
- Zone context injected into Nemotron prompt alongside vision evidence
- Nemotron returns priority + supervisor_action + rationale + steps[]
- Full AnalyzeResponse saved to TiDB audit table
- Response returned to React: zone compliance panel + Nemotron card + evidence cards

The full system architecture diagram is at docs/system_architecture.svg in the repository — 4 layers with colour-coded flow arrows.

4. AI PIPELINE — DETAILED FLOW

Step 1 — GPT-4o Vision

Worker photo is base64-encoded and sent as a data URL. Prompt requests response_format: json_object with fields:

- hard_hat: pass | fail | unclear
- hi_vis: pass | fail | unclear
- fatigue_risk: low | medium | high
- evidence[]: [{region, observation, severity}]
- overall_risk: low | medium | high | critical

Step 2 — Zone Compliance Engine

zone_service.py loads zones.json and compares detected PPE against zone required_ppe list. Each item gets ComplianceStatus:

- compliant — required and detected as pass
- non_compliant — required but detected as fail or unclear
- not_required — not in this zone's required_ppe list

Step 3 — Nemotron Reasoning

Receives vision evidence + zone compliance + Ontario Reg 854 context. Returns:

- priority: stop_work | intervene | monitor | clear
- supervisor_action: one-sentence directive
- rationale: why this priority was assigned
- steps[]: ordered list of concrete supervisor actions

Fallback chain: NVIDIA Nemotron API → OpenAI GPT-4o-mini → mock response.

Step 4 — Audit Persistence

Every scan saved to TiDB Cloud (northernguard cluster). Table: id, image_filename, hard_hat, hi_vis, fatigue_risk, priority, supervisor_action, zone_id, zone_name, zone_overall, vision_json, nemotron_json, zone_json, provider_vision, provider_nemotron, created_at.

5. ZONE-AWARE COMPLIANCE ENGINE

Five mine zones defined in backend/config/zones.json, each referencing specific Ontario Regulation 854 sections:

Zone ID	Name	Required PPE	Fatigue Check	Regulation
surface	Surface / Yard	hi_vis	No	Reg 854 s.79
open_pit	Open Pit	hard_hat, hi_vis	Yes	Reg 854 s.79, s.81
underground_entry	Underground Entry	hard_hat, hi_vis	Yes	Reg 854 s.79, s.81
active_stope	Active Stope	hard_hat, hi_vis	Yes	Reg 854 s.79, s.81, s.105
processing_plant	Processing Plant	hard_hat, hi_vis	Yes	Reg 854 s.79, s.81

Surface / Yard only requires hi-vis. A worker without a hard hat at the surface yard is compliant. The same worker at the active stope is non-compliant. This zone-awareness is what makes the system practically useful vs. generic PPE detectors.

6. TECH STACK

Layer	Technology	Notes
IDE	Cursor	Hackathon sponsor tool
Vision AI	OpenAI GPT-4o	response_format: json_object · data URL image input
Reasoning AI	NVIDIA Nemotron	integrate.api.nvidia.com · GPT-4o-mini fallback
Backend	FastAPI + Uvicorn	Python 3.12 · multipart Form + File upload
Schema validation	Pydantic v2	AnalyzeResponse, ZoneCompliance, ScanSummary
Database (prod)	TiDB Cloud	MySQL-compatible · northernguard cluster
Database (local)	SQLite	SQLAlchemy · auto-migration on startup
Frontend	React + Vite	TypeScript · Tailwind CSS
UI scaffold	Lovable	Industrial dark theme
Container	Docker	Single container: FastAPI serves API + built React dist
Deploy	Render	render.yaml blueprint included
Notebook	Jupyter	northern_shift_guard.ipynb — pipeline prototype
Safety refs	Local text files	data/safety_refs/ — Ontario Reg 854, OSHA PPE

7. PROJECT FILE STRUCTURE

Path	Description
backend/main.py	FastAPI app — /api/analyze, /api/zones, /api/scans
backend/vision_service.py	GPT-4o vision — Replicate fallback — mock fallback
backend/zone_service.py	Zone compliance engine — loads zones.json
backend/nemotron_service.py	NVIDIA Nemotron reasoning — GPT-4o-mini fallback
backend/db.py	SQLAlchemy TiDB/SQLite — save_scan(), get_recent_scans()
backend/schemas.py	Pydantic models: AnalyzeResponse, ZoneCompliance, ScanSummary
backend/settings.py	Pydantic-settings: env vars, provider routing, DB URL
backend/prompts.py	Prompt templates for vision and Nemotron
backend/config/zones.json	Zone definitions: required PPE, regulation refs
frontend/src/App.tsx	Root component — zone state — analyzeImage handler
frontend/src/components/ImageUpload.tsx	Zone selector grid + dropzone
frontend/src/components/AnalysisResult.tsx	PPE cards + zone compliance panel + Nemotron action
frontend/src/components/ScanHistory.tsx	Audit trail tab
frontend/src/api.ts	analyzeImage(file, zone) — fetchScans() — fetchZones()
frontend/src/types.ts	TypeScript interfaces: AnalyzeResponse, ZoneCompliance, etc.
notebooks/northern_shift_guard.ipynb	Full pipeline prototype: vision - zone - reasoning
data/safety_refs/	Ontario Reg 854, OSHA PPE, Northern ON mining context
sample_images/	Demo: fail_missing_hardhat.jpg, pass_compliant.jpg, fatigue_tired_operator.jpg
docs/devpost_story.md	Full Devpost project story
docs/system_architecture.svg	System architecture diagram (4-layer pipeline)
run.sh	Quick-start: backend (8000) + frontend (5173) in parallel
Dockerfile	Production single-container build
render.yaml	Render deployment blueprint
.env.example	Required env vars template

8. API REFERENCE

Method	Endpoint	Description
GET	/health	Health check — returns project name and team
GET	/api/zones	All zone definitions with required_ppe and regulation refs
POST	/api/analyze	Main endpoint — image (File) + zone (Form) — returns AnalyzeResponse
GET	/api/scans	Scan history from audit log — optional limit param (default 20)

POST /api/analyze — Response Shape:

```
scan_id · vision {ppe: {hard_hat, hi_vis}, fatigue: {risk, score}, evidence[], overall_risk} ·  
zone_compliance {zone_id, zone_name, overall, items[{ppe_item, required, detected, status}],  
regulation} · nemotron {priority, supervisor_action, rationale, steps[]} · provider {vision,  
nemotron} · image_filename · created_at
```

9. SETUP & RUNNING

Prerequisites

- Python 3.10+
- Node.js 18+
- OPENAI_API_KEY (required)
- NVIDIA_API_KEY (optional — falls back to GPT-4o-mini)
- TIDB_DSN (optional — falls back to SQLite)

Quick Start

```
./run.sh
```

run.sh activates venv, installs Python deps, starts FastAPI on :8000 and Vite on :5173. Ctrl+C kills both.

Manual Setup

```
cp .env.example .env # fill in OPENAI_API_KEY at minimum
cd backend && source ../venv/bin/activate && pip install -r requirements.txt
uvicorn main:app --reload --port 8000
# New terminal: cd frontend && npm install && npm run dev
```

Variable	Required	Default	Description
OPENAI_API_KEY	Yes	—	OpenAI key for GPT-4o vision + fallback reasoning
VISION_PROVIDER	No	openai	'openai' 'replicate' 'mock'
NVIDIA_API_KEY	No	—	NVIDIA key for Nemotron reasoning
NEMOTRON_PROVIDER	No	auto	'nvidia' 'openai' 'mock'
TIDB_DSN	No	—	TiDB Cloud DSN — unset uses SQLite
CORS_ORIGINS	No	http://localhost:5173	Allowed CORS origins

10. DEMO WALKTHROUGH

Key judge moment: same photo, different zone = different compliance verdict.

Step	Action	Expected Result
1	Select zone: Active Stope	Hard hat + hi-vis required. Fatigue check enabled (s.79, s.81, s.105)
2	Upload: fail_missing_hardhat.jpg	Hard hat: FAIL — Zone: non-compliant — Nemotron: stop_work
3	Change zone to: Surface / Yard	Hard hat not required at surface — same photo now compliant for hard hat
4	Upload: pass_compliant.jpg (Open Pit)	Hard hat + hi-vis: PASS — Zone: compliant — Nemotron: clear
5	Upload: fatigue_tired_operator.jpg	Fatigue: high — Nemotron: monitor or intervene with fatigue rationale
6	Open Audit Trail tab	All scans shown: zone, PPE result, Nemotron action, timestamp

11. JUDGING CRITERIA MAPPING

Criterion	How Northern Shift Guard Addresses It
Problem Relevance	Direct response to Northern Ontario mining — MOL audit risk, new worker fatigue, zone-specific PPE under Reg 854
Solution Design	4-layer pipeline (detect - zone compliance - reason - audit). Rules grounded in actual Ontario Reg 854 sections
Working Prototype	Fully functional: FastAPI + React UI + GPT-4o + Nemotron + TiDB audit. Demo-ready locally
Impact	Every shift-start decision becomes explainable and auditable. Scales to webcam checkpoints and batch crew scanning
Presentation	Pitch deck v2, Devpost story, 60s demo script, architecture diagram, and this project report
Responsible Design	Fatigue screening labeled visual aid only — not medical diagnosis. No biometric data stored. Human review via audit
Originality	Zone-aware compliance is novel — GPT-4o + Nemotron exist independently; combining with zone-specific Reg 854

12. WHAT WE LEARNED

Zone-aware compliance is the real differentiator

Generic PPE detectors exist. What mines need is a system that knows the difference between surface and underground — and adjusts both compliance verdict and supervisor guidance.

GPT-4o vision + structured JSON is production-ready

Evidence quality (region + observation + severity) far exceeds what a YOLO classifier alone provides for safety screening.

Explainability is a requirement, not a feature

In regulated industries, every layer must answer: why did the system say that? Vision evidence, zone compliance rationale, and Nemotron step-by-step actions all serve this.

Pivot fast when a dependency breaks

Replicate LLaVA-13B returned a 422 error on day-of. Switching to GPT-4o improved quality and enabled structured JSON output — the pivot made the system better.

Inference-time API chaining vs. training from scratch

This is not a train-from-scratch ML pipeline. It is inference-time API composition: GPT-4o (pre-trained) + zone rules engine + Nemotron (pre-trained) + TiDB. The innovation is in the composition and the zone compliance layer.

13. NEXT STEPS

Feature	Description
Webcam / real-time capture	Scan workers as they pass a checkpoint, not just via upload
Batch shift scanning	Process an entire crew at once before cage descent
TiDB analytics dashboard	Shift-over-shift compliance trends, zone risk heatmaps, flagged-worker patterns
Expand PPE types	Gloves, safety glasses, respirators, metatarsal boots — per-zone config
WSIB / MOL report export	Auto-generate incident documentation from TiDB audit records
Wearable integration	Connect to proximity/gas sensors to enrich fatigue context
Multi-site deployment	Zone config per mine site — Sudbury, Timmins, Kirkland Lake

14. SUBMISSION NOTE & GITHUB

NOTE: Could not submit project file archive to Devpost due to file size. Full source code added to GitHub instead.

The complete project archive could not be attached to Devpost. The combined archive — including Python virtual environment (venv/), Node.js packages (node_modules/), and sample media — exceeds the Devpost file size limit.

The full source code is available on GitHub (link on Devpost project page). The repository includes:

- Complete backend source (backend/*.py, backend/config/)
- Complete frontend source (frontend/src/)
- Jupyter notebook prototype (notebooks/northern_shift_guard.ipynb)
- Safety reference data (data/safety_refs/)
- Zone configuration (backend/config/zones.json)
- Sample images (sample_images/)
- Submission media (submission_media/)
- System architecture diagram (docs/system_architecture.svg)
- Devpost project story (docs/devpost_story.md)
- Pitch deck (Northern_Shift_Guard_Pitch_v2.pptx)
- Docker + Render deployment config (Dockerfile, render.yaml)
- Environment variable template (.env.example)
- Quick-start script (run.sh)

.env with API keys is gitignored and was never committed. All keys are injected at runtime via environment variables.